

Mardik v2 — Dossier de conception, Chantier 1 : la chaîne LLMOps

Livrable de conception (porte d'entrée du brief) — version de travail, 2026-09-16. Auteur : Eramalingam GANDAKUMAR (solo). Brief : Elbby Skermine AGWU, 14/09/2026. Périmètre : Chantier 1 seul. Les pièces du Chantier 2 (schéma de la boucle d'observabilité, tableau de pilotage) font l'objet d'un livrable séparé après validation de celui-ci ; leurs points de raccord sont dessinés ici.

Sommaire

1. Synthèse du besoin	2
1 bis. Compléments : contexte, valeur, critère de couverture, vue d'ensemble	2
2. Contrat d'API v2	5
2.1 Principes du contrat	5
2.2 Ressources	5
2.3 POST /v2/analyses — requête	5
2.4 POST /v2/analyses — réponse 200	6
2.5 Le score de confiance — ce que clauses[].confiance et score_confiance promettent	6
2.6 Idempotence et nouvelles tentatives — un appel au modèle est un appel payé	7
2.7 Erreurs explicites — un code, un message, une cause	7
2.8 Ce qui prouve le contrat	8
3. Schéma de la chaîne LLMOps	9
3.0 Avant la fusion : commit, branche, pull request	11
3.1 Comment lire le schéma	12
3.2 Les six étapes de la chaîne	13
3.3 « Artefact étiqueté » — ce que le brief demande, ce que la chaîne produit	13
3.4 La production — où v1 ne casse jamais	14
3.5 Points de raccord avec le Chantier 2 — dessinés, pas conçus	14
3.6 Ce qui prouve la chaîne	15
3.7 Où vivent les secrets — et correspondance avec docs/exploitation.md	15
4. Le manifeste de version	15
4.1 Deux couches : le Bundle fourni, et ce que le manifeste y ajoute	16
4.2 Où il vit — cinq endroits, une seule source	17
4.3 Deux horloges, un numéro : ce qui change quoi	17
4.4 Retrouver, à 3 h du matin, ce qui tourne	18
4.5 Intégrité — ce qui est fait, ce qui est dit	18
4.6 Choix d'outils non triviaux — ce qui est retenu, ce qui est écarté, pourquoi	18
5. Hypothèses et limites	19
5.1 Ce qui est un fait, et d'où il vient	19
5.2 Ce qui est supposé — et ce qui se passe si c'est faux	19
5.3 Hors périmètre (rappel)	20
5.4 Ce que ce dossier ne fait pas, et ses limites	20
6. Critères RNCP couverts	20
Annexe A — Réponses aux questions de cadrage du brief (Chantier 1)	21
 Glossaire	23

1. Synthèse du besoin

Trois juristes, ~400 contrats par mois collés dans leur outil documentaire qui appelle l'API ; la v1 coupe en silence au-delà de 16 000 caractères et la dernière livraison, faite à la main, a coûté une demi-journée. La v2 doit analyser des contrats entiers avec un score de confiance, sans toucher à /v1, livrée par une chaîne — jamais à la main. Le détail (contexte, valeur, critère de couverture, vue d'ensemble) est en section 1 bis.

Besoin fonctionnel (ce que v2 fait).

- Analyser un contrat **entier**, collé en texte comme aujourd'hui, et restituer les types de clauses présents dans le vocabulaire déjà partagé par la v1 et les juristes (durée, résiliation, droit applicable...).
- Accompagner l'analyse d'un **score de confiance par clause et global**, calibré : sa valeur correspond à une exactitude mesurée sur les contrats de référence fournis par la direction juridique, pas à une impression du modèle.
- Répondre par une **erreur explicite** (code + message) à toute entrée invalide ou hors limites — là où v1 coupait, v2 refuse à voix haute.
- Exposer tout cela sur /v2, **sans modifier le contrat de /v1**, que le client intégré continue d'appeler tel quel — il n'y a pas de budget pour le refaire.

Contraintes (ce qui doit rester vrai).

- **Performance (direction juridique)** : latence **p95 < 8 s**, **contrats longs compris** ; coût **< 0,15 € par analyse** en moyenne.
- **Livraison (CTO)** : toute fusion produit une version étiquetée (code + modèle + configuration), testée par des gates bloquants ; un gate en échec bloque ; aucune livraison manuelle, y compris pour un correctif.
- **Bascule** : progressive (canary), pilotée par les métriques ; retour arrière immédiat — décision humaine en un clic, exécutée par la chaîne et tracée.
- **Observabilité** : latence, coût, taux d'erreur, distribution du score, part de trafic v1/v2 — émis dès le Chantier 1 dans le format que le tableau de bord fourni sait lire, exploités au Chantier 2.
- **Existant** : télémétrie, tableau de bord, surveillance et tests d'intégration issus de la remédiation sont fournis fonctionnels — on construit dessus, on ne les refait pas.

Hors périmètre de la v2 (dit, pas oublié).

- La **traduction en anglais** et le **chatbot** demandés « pendant qu'on y est » : absents de la note du CTO, incompatibles avec le budget de 8 s et 0,15 € par analyse, et un autre besoin que l'analyse de clauses. À instruire à part.
- Le découpage des documents au-delà du plafond de taille (chunking) : itération ultérieure — agréger des scores calculés sur des fragments est un problème d'évaluation non résolu ici ; au-delà du plafond, v2 refuse explicitement.
- Toute modification fonctionnelle de v1 ; seule son **instrumentation** est autorisée.
- Le tableau de bord, les trois boucles et le journal de pilotage : Chantier 2, après validation de celui-ci ; points de raccord en section 3.

1 bis. Compléments : contexte, valeur, critère de couverture, vue d'ensemble

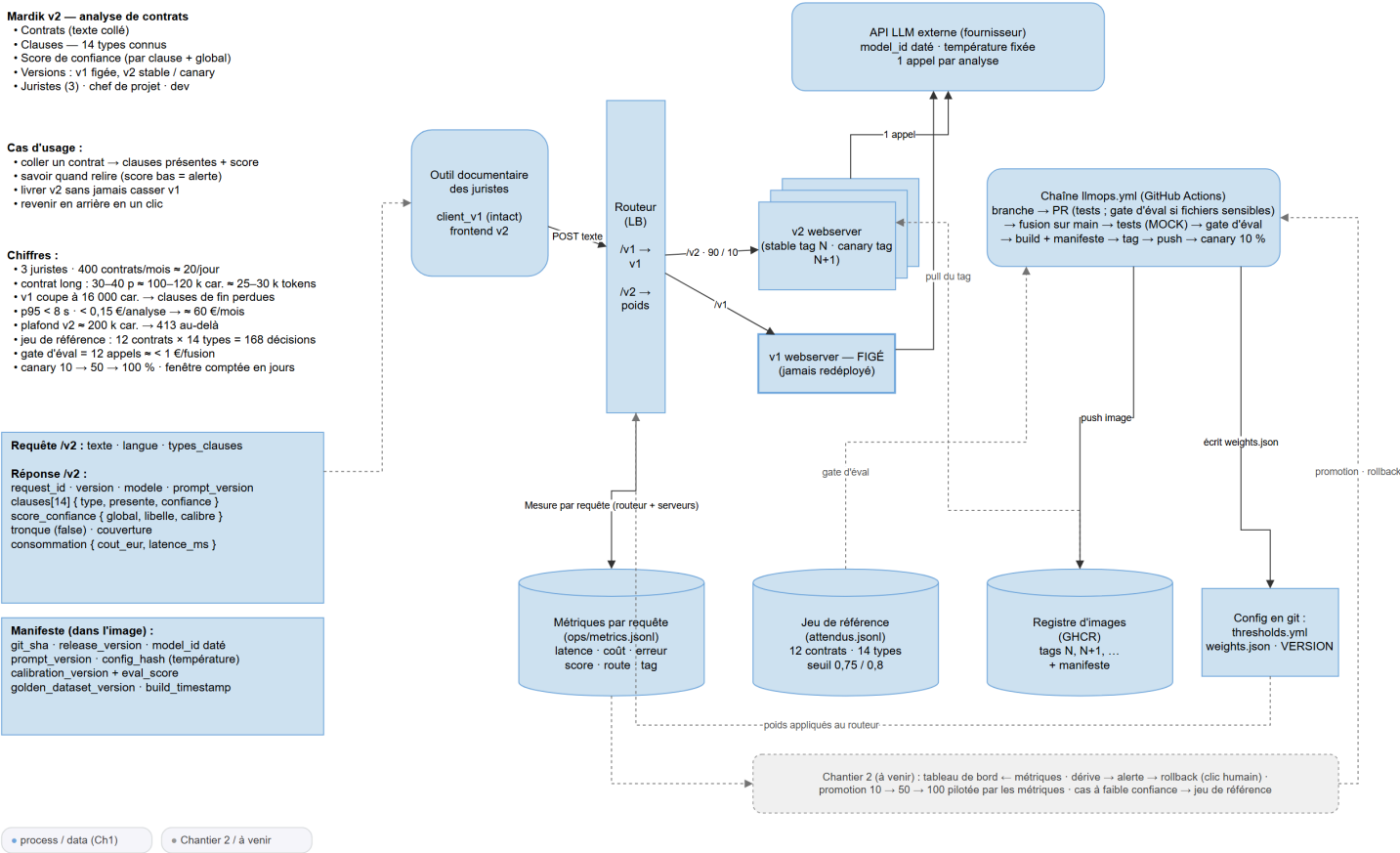
Contexte et utilisateurs. Les trois juristes de Mardik collent chaque jour le texte d'un contrat dans leur outil de gestion documentaire, qui appelle l'API (`client_v1`) et rend la liste des types de clauses présents — environ 400 contrats par mois. La v1, un service unique appelant une API LLM externe, a deux défauts distincts. Un défaut **produit** : au-delà de 16 000 caractères le texte est coupé avant l'appel au modèle ; l'API le signale (`tronque`), mais pour le juriste l'effet est silencieux — les clauses de fin (résiliation, pénalités, droit

applicable) manquent sans alerte, et les contrats longs (30 à 40 pages : contrats-cadres, appels d'offres, distribution) ne lui sont plus confiés ; et aucune réponse ne dit quand relire. Un défaut **d'exploitation** : la dernière livraison, faite à la main, a coûté une demi-journée d'indisponibilité, sans version identifiable ni retour arrière. La note du CTO (« Plus jamais ça ») lie les deux.

Valeur. Pour les juristes : retrouver les contrats longs dans l'outil et savoir **quand relire** — un score bas alerte, un score haut est fiable. Pour le chef de projet : **constater** que la v2 est meilleure avant de la généraliser, et revenir en arrière **immédiatement**, sans attendre un correctif. Pour l'équipe : ne plus jamais livrer à la main.

Ce qui dira que le besoin est couvert. Les cinq tests d'acceptance « chaîne LLMOps », rejoués par la chaîne à chaque fusion ; et, sur les 12 contrats de référence, une note par contrat au-dessus du seuil fixé par la direction juridique (0,75 ; 0,8 pour les contrats longs) — là où la v1 échoue par construction sur les trois contrats longs.

Vue d'ensemble du système. Avant le contrat (section 2) et la chaîne (section 3), le système entier sur une page, à la manière d'un tableau blanc de conception : à gauche ce que l'on manipule, les cas d'usage et les ordres de grandeur ; au centre le chemin d'une requête, du poste du juriste au fournisseur de modèle ; en bas ce qui est stocké et versionné, et la chaîne qui livre.



Source éditable : `schemas/schema-systeme-mardik.drawio`.

2. Contrat d'API v2

Question : comment `/v2` répond-il sans jamais toucher au contrat de `/v1` ? **Réponse courte :** un routeur séparé, deux horloges de version qui ne se confondent jamais (le contrat dans l'URL, le comportement dans le manifeste), un score de confiance calibré plutôt qu'une confiance affichée telle quelle, et une erreur explicite à la place de tout `200` dégradé. Le détail section par section suit.

2.1 Principes du contrat

- **Le contrat est le code des schémas.** Les schémas Pydantic de requête et de réponse sont la source unique ; la spécification OpenAPI (`/v2/openapi.json`) en est générée. Aucune documentation écrite à part qui pourrait diverger.
- **Un routeur par version.** `/v1` et `/v2` sont deux routeurs distincts, deux jeux de schémas. Le routeur `/v1` n'est pas modifié ; il est seulement **instrumenté** (journaux, métriques) et son comportement est prouvé inchangé par le rejeu de `scripts/client_v1.py` comme gate de la chaîne.
- **Deux horloges de version, jamais confondues.**
 - *Horloge du contrat :* la version majeure est dans l'URL (`/v2`). Un changement incompatible (champ retiré, sens modifié, code d'erreur changé) impose `/v3` et une période de maintien de `/v2` . Un ajout compatible (champ optionnel nouveau) ne change pas l'URL.
 - *Horloge du comportement :* `release_version` (semver), `model_id` (identifiant daté du fournisseur), `prompt_version` , `calibration_version` . Un prompt modifié, un modèle mis à jour ou une calibration refaite produisent une **nouvelle release**, pas un nouveau contrat : la forme des réponses ne bouge pas, leur contenu peut.
- **Toute réponse est signée par sa version.** Succès comme erreur, chaque réponse porte `request_id` (corrélation) et l'en-tête `X-Mardik-Release: <release_version>` . C'est la preuve demandée par le brief — « identifier quelle version sert quel trafic » — à la granularité de la requête. L'en-tête est aussi ajouté aux réponses de `/v1` : ajout non rompant, vérifié par le rejeu de `client_v1.py` .
- **Le même langage que v1 et que les juristes.** Champs nommés en français (`texte` , `clauses` , `modele` , `version` , `tronque` , `score_confiance`), types de clauses pris dans le vocabulaire existant (`TYPES_CLAUSES`) : un seul modèle de domaine entre le métier et le code, pas de traduction à l'interface.
- **Jamais un 200 dégradé.** Une réponse `200` est complète ou n'est pas : `v2` n'a pas de mode « analyse partielle » . C'est le renversement exact du défaut de `v1`.

2.2 Ressources

Méthode et route	Rôle
POST <code>/v2/analyses</code>	analyser un contrat entier ; la ressource du chantier
GET <code>/v2/analyses/{request_id}</code>	relire une analyse déjà produite (rejeu sans nouvel appel au modèle)
GET <code>/v2/version</code>	renvoyer le manifeste de la release qui répond (section 4)
GET <code>/v2/health</code>	vivacité et disponibilité du fournisseur de modèle — lu par le routeur avant d'envoyer du trafic canary

2.3 POST `/v2/analyses` — requête

`application/json` , la même forme que `/v1` — c'est ainsi que l'outil documentaire des juristes travaille (le texte est collé) :

```
{ "texte": "<texte intégral du contrat>", "langue": "fr", "types_clauses": [] }
```

- `texte` obligatoire, au moins 20 caractères (comme `v1`) ; `langue` optionnel (`fr` par défaut) ; `types_clauses` optionnel — restreindre l'analyse à certains types ; vide = les 14 types connus.
- Le dépôt de fichiers (PDF) n'est **pas** dans la `v2.0` : les juristes collent le texte, l'extraction reste du côté de leur outil documentaire. Une itération ultérieure pourra l'ajouter sans changer ce contrat.

Limites, vérifiées **avant** tout appel au modèle : taille maximale du texte (`max_caracteres`, valeur de travail en section 5, au-dessus des 40 pages du besoin). Une limite franchie produit une erreur explicite, jamais une troncature.

En-têtes : `Idempotency-Key` (recommandé, voir 2.6) · `X-API-Key` (obligatoire hors environnement de démonstration).

2.4 POST /v2/analyses — réponse 200

```
{
  "request_id": "req_01J8ZK3Q7X",
  "version": "v2.0.0",
  "modele": "<identifiant daté du fournisseur>",
  "prompt_version": "analyse-v2.0",
  "clauses": [
    { "type": "durée",          "presente": true,  "confiance": 0.93 },
    { "type": "résiliation",   "presente": true,  "confiance": 0.88 },
    { "type": "droit applicable", "presente": true,  "confiance": 0.81 },
    { "type": "exclusivité",    "presente": false, "confiance": 0.90 }
  ],
  "score_confiance": {
    "global": 0.84,
    "libelle": "haute",
    "calibre": true,
    "calibration_version": "cal-v1",
    "composantes": { "schema_valide": 1.0, "couverture": 1.0, "auto_evaluation": 0.79 }
  },
  "tronque": false,
  "couverture": { "caracteres_total": 84213, "caracteres_analyses": 84213 },
  "consommation": { "tokens_entree": 21053, "tokens_sortie": 210, "cout_eur": 0.06, "latence_ms": 4210 }
}
```

- `clauses` couvre **les 14 types connus**, chacun avec `presente` et sa `confiance` : une clause absente est une information (un contrat sans « droit applicable » est un signal pour le juriste), et chaque décision est une prédiction binaire calibrable contre le jeu de référence. Le client `v1` ne recevait que la liste des types présents ; le frontend `v2` peut l'afficher de la même façon en filtrant `presente`.
- `tronque` vaut toujours `false` en `v2` par construction (appel unique, plafond explicite) ; il est conservé pour que le contraste avec `v1` soit **mesurable** dans les mêmes journaux, pas seulement affirmé.
- `couverture` rend visible ce qui a été lu ; le champ existe pour que le contrat survive à une itération avec découpage sans changer de forme.
- `consommation` porte coût et latence **par requête** : deux des cinq signaux du Chantier 2, produits ici, écrits dans la `Mesure` de la télémétrie fournie, jamais recalculés ailleurs.

2.5 Le score de confiance — ce que `clauses[].confiance` et `score_confiance` promettent

- **Par clause.** Le modèle est interrogé en un appel, avec une sortie structurée : pour chacun des 14 types, présent ou non, et une auto-évaluation brute. Cette valeur brute est ensuite **calibrée** par une courbe

(régression isotonique) apprise hors ligne sur le jeu de référence : les 12 contrats de `attendus.jsonl` × 14 types = 168 décisions dont on connaît la vérité (`clauses_attendues`). Après calibration, `confiance = 0,8` signifie « juste environ 8 fois sur 10 » — mesuré, pas ressenti. La courbe est un artefact versionné (`calibration_version`), embarqué dans la release, listé dans le manifeste.

- **Global.** `score_confiance.global` combine la moyenne des confiances calibrées et deux composantes de bon fonctionnement, `schema_valide` (la sortie respecte-t-elle le schéma ?) et `couverture` (part du texte analysée) — exposées dans `composantes` pour que le score soit **explicable**.
- **Le libellé, pour le juriste.** `libelle` dérive du score global par des bornes en configuration versionnée (valeurs de travail : `haute` ≥ 0,8 · `moyenne` [0,5 ; 0,8[· `basse` < 0,5), et porte une règle de décision, pas seulement une couleur : `haute` → valider ; `moyenne` → vérifier les clauses dont la confiance est sous 0,7 ; `basse` → relire entièrement. C'est le « niveau de certitude » demandé par le métier, sous le nom que le frontend affichera.
- `calibre` vaut `false` quand le contrat sort du domaine couvert par le jeu de référence (langue, type de contrat) : le score est renvoyé mais **déclaré non fiable** — jamais un chiffre rassurant sur un cas inconnu.
- **La note d'évaluation n'est pas le score.** La note d'un contrat, celle du gate, mesure l'exactitude de la liste rendue contre `clauses_attendues` (F1 sur les types présents) et se compare au `seuil_note` du contrat (0,75 ; 0,8 pour les longs). Le score de confiance prédit cette exactitude à l'exécution, sans connaître la vérité. Deux objets, deux usages : la note bloque une livraison ; le score dit au juriste quand relire.
- Ce que le score ne promet pas : il quantifie la fiabilité de ce qui est renvoyé, sur le domaine où il a été calibré. Dit ici, redit en section 5.

2.6 Idempotence et nouvelles tentatives — un appel au modèle est un appel payé

- **Idempotency-Key** : un identifiant fourni par le client. Si la même clé est reçue une seconde fois (réseau coupé, client qui réessaie), le serveur renvoie la réponse déjà produite avec l'en-tête `Idempotent-Replayed: true` — **sans nouvel appel au modèle**, donc sans second coût ni seconde latence. Conservation : 24 h (valeur de travail).
- **Réessais côté serveur, vers le fournisseur** : au plus 2, uniquement sur 429, 5xx et délai dépassé, avec attente croissante, et **dans le budget de latence total** de la requête. Une réponse 200 du fournisseur mais inexploitable (JSON invalide) compte comme une tentative : elle a été payée.
- **Budget de latence** : un délai total par requête (valeur de travail en section 5) ; à son terme, 504, jamais une réponse partielle.

2.7 Erreurs explicites — un code, un message, une cause

Toutes les erreurs partagent une forme unique, celle des *problem details* (RFC 9457), avec deux champs ajoutés :

```
{
  "type": "urn:mardik:error:document-too-large",
  "title": "Document trop long",
  "status": 413,
  "detail": "Le document fait 212 400 caractères ; le maximum accepté est 150 000.",
  "code": "DOCUMENT_TOO_LARGE",
  "request_id": "req_01J8ZK3Q7X",
  "limit": { "max_chars": 150000 }
}
```

Statut	code	Quand	Ce que le client peut faire
400	MALFORMED_REQUEST	JSON invalide, champ obligatoire absent, deux formes d'entrée à la fois	corriger la requête
401	UNAUTHENTICATED	clé d'API absente ou invalide (hors démo)	fournir une clé
413	DOCUMENT_TOO_LARGE	texte au-dessus du plafond <code>max_caracteres</code> — le cas que v1 coupait	réduire le texte ; le plafond est dans <code>limit</code>
422	TEXTE_INVALIDE	texte absent, vide ou sous 20 caractères ; <code>types_clauses</code> hors vocabulaire	fournir un texte de contrat ; les types connus sont dans <code>limit</code>
429	RATE_LIMITED	trop de requêtes ; en-tête <code>Retry-After</code>	attendre puis réessayer avec la même <code>Idempotency-Key</code>
502	MODEL_RESPONSE_INVALID	le fournisseur a répondu, mais la sortie est inexploitable après les réessais	réessayer plus tard ; l'événement est tracé
503	MODEL_UNAVAILABLE	fournisseur injoignable ; en-tête <code>Retry-After</code>	réessayer plus tard
504	MODEL_TIMEOUT	budget de latence épuisé	réessayer, éventuellement avec un document plus court

Règles : aucune erreur `500` attendue — si elle survient, c'est un défaut, tracé par `request_id`. Aucun message d'erreur n'expose de détail interne (chemin, pile d'appels, clé). Les erreurs `4xx` sont produites **avant** tout appel au modèle : une requête invalide ne coûte rien.

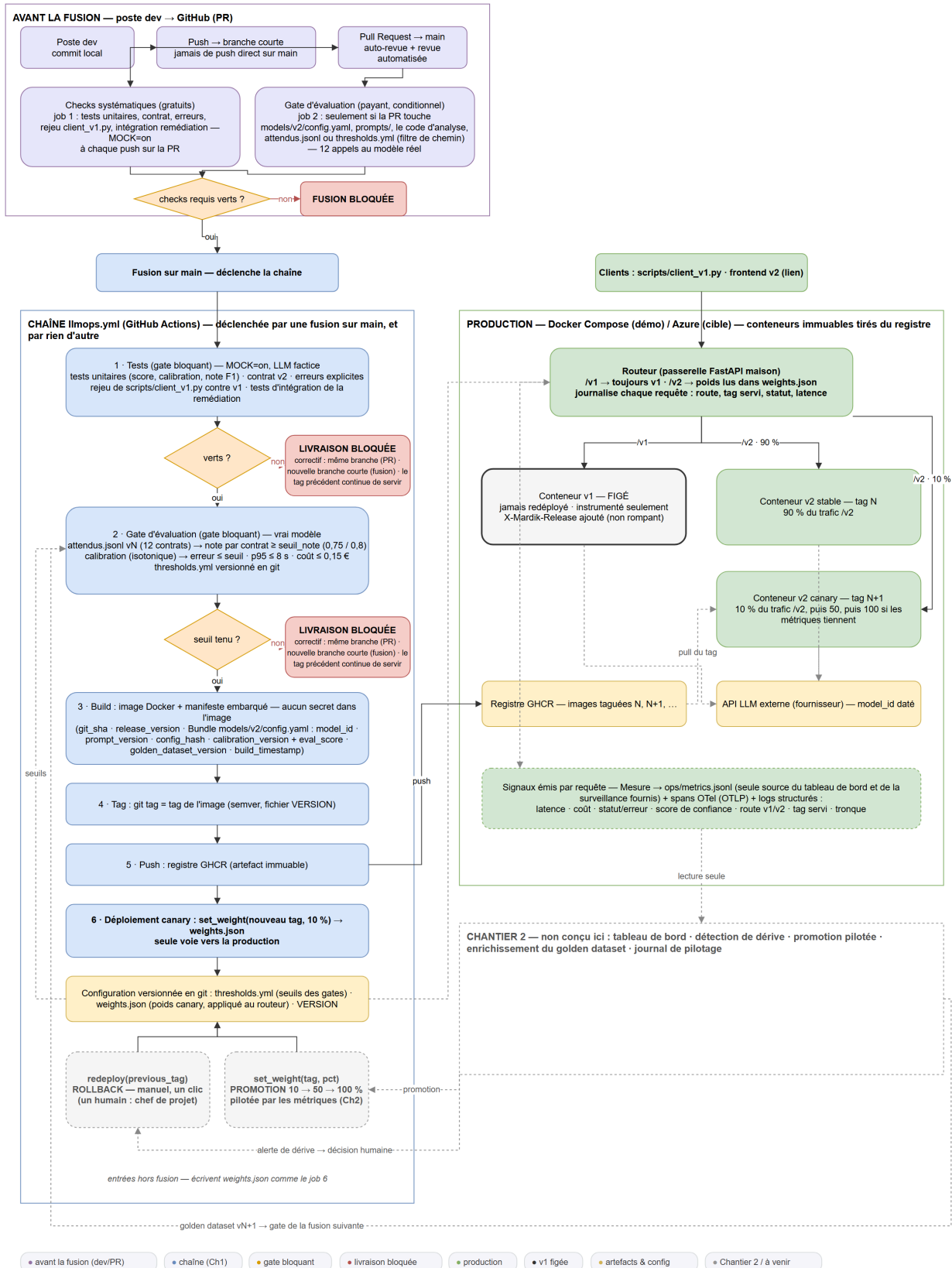
2.8 Ce qui prouve le contrat

- `tests/test_v2_contract.py` — un document long produit une réponse conforme au schéma avec un `confidence.score` (test d'acceptance 1).
- `tests/test_v2_errors.py` — chaque ligne du tableau 2.7 a un test qui provoque l'erreur et vérifie code, statut et forme (test d'acceptance 2).
- `scripts/client_v1.py` rejoué contre v1 instrumentée, comme gate de `11mops.yml` (test d'acceptance 3).
- `tests/test_score.py`, `tests/test_note.py` — tests unitaires, sans modèle : le score composite, la courbe de calibration (un score brut donné → le score calibré attendu), la note F1, l'analyse des types de clauses.
- `ressource/attendus.jsonl` (12 contrats, seuils) est la fixture du gate d'évaluation ; `ressource/exemple_de_contrat.txt` celle des tests de contrat.
- Les exemples de ce chapitre sont les fixtures de ces tests : le dossier et les tests ne peuvent pas diverger.

3. Schéma de la chaîne LLMOps

Question : qu'est-ce qui empêche une livraison manuelle ou une régression de `/v1`, du premier commit jusqu'au trafic réel ? **Réponse courte :** deux gates bloquants avant même de construire une image (des checks gratuits systématiques, un gate d'évaluation payant et conditionnel — section 3.0), deux gates bloquants de nouveau à la fusion sur le code réellement fusionné (section 3.2), un artefact immuable une fois taggé, et `/v1` qui n'est jamais redéployé par cette chaîne, seulement instrumenté (section 3.4). Le schéma montre les deux temps : avant la fusion (poste dev → PR, encadré violet), puis la chaîne elle-même.

Mardik v2 — Chaîne LLMops (Chantier 1) : de la branche à la fusion, puis gates bloquants → build → canary



Source éditable : [schemas/schema-chaîne-llmops.drawio](#).

3.0 Avant la fusion : commit, branche, pull request

Répond aux trois questions posées par le formateur le 18/09 sur ce dossier : quelle stratégie de branche, quels tests tournent où et pourquoi pas tous à chaque fois, quand reteste-t-on un changement de modèle ou de prompt.

Pas de branche `dev` partagée. Une branche courte par changement, jamais de push direct sur `main` (protection de branche GitHub). Raison : la section 4.3 établit que *chaque fusion sur `main` = une release* ; une branche `dev` intermédiaire casserait cette correspondance — on ne saurait plus quelle fusion produit quelle version.

Étape	Déclenché par	Ce qui tourne	Pourquoi (pas tout, pas partout)
Commit	poste dev	rien (local)	—
Push vers la branche	GitHub	rien	la branche seule ne déclenche aucun job — coûterait à chaque frappe, pour rien avant qu'une PR existe
Ouverture / mise à jour de la PR	GitHub Actions	job 1 — tests unitaires (score, calibration, note F1), contrat v2, erreurs explicites, rejeu de <code>scripts/client_v1.py</code> contre v1, tests d'intégration de la remédiation (<code>MOCK=on</code>)	gratuit (LLM factice) : peut tourner à chaque push, aucune raison de s'en priver
Ouverture / mise à jour de la PR — filtre de chemin	GitHub Actions	job 2 , le gate d'évaluation (12 appels au modèle réel, payant) — seulement si la PR touche <code>models/v2/config.yaml</code> , <code>prompts/</code> , le code d'analyse, ou les entrées du gate lui-même : <code>attendus.jsonl</code> , <code>thresholds.yaml</code> (une PR d'enrichissement ou d'ajustement de seuil change ce que le gate mesure)	un changement de prompt ou de modèle change le comportement (section 4.3) : il doit être revérifié avant la fusion, pas découvert après. Une PR qui ne touche à rien de tout ça (refactoring, documentation) ne le déclenche pas — le payer serait mesurer ce qui n'a pas changé
Fusion sur <code>main</code> (checks requis verts + relecture : auto-revue du diff, assumée comme telle en solo, plus une revue automatisée sur chaque PR — Claude Code GitHub Action , authentifiée par le jeton d'abonnement de l'auteur (<code>CLAUDE_CODE_OAUTH_TOKEN</code> , généré par <code>claude setup-token</code> ; pas de clé d'API, pas de financement à demander) ; repli : la revue de code GitHub Copilot)	déclenche <code>llmops.yml</code> 1	job 1 et job 2, systématiquement, sans condition (section 3.2)	le code réellement fusionné peut différer de celui de la PR si <code>main</code> a bougé entre-temps ; l'exigence « aucune livraison manuelle » interdit de se fier au seul résultat de la PR

Seul sur ce projet, la relecture n'est pas une revue à deux ; elle est dite telle quelle. Le second regard est automatisé : un outil relit le diff de chaque PR et commente avant la fusion. Il ne remplace pas un gate — un commentaire ne bloque rien — mais il rend visible ce qu'une seule paire d'yeux rate.

Le filtre de chemin qui déclenche le job 2 sur une PR est le même que celui qui, à la section 4.3, dit qu'un changement de prompt, de modèle ou de configuration incrémente la version MINOR : une PR qui

déclenche le gate d'évaluation est, par construction, une PR qui prépare une MINOR.

Trois façons de déclencher le gate d'évaluation, et celle retenue. En séance du 18/09, le formateur a posé trois réponses à la tension coût des tests / vitesse de livraison / qualité : (a) un déclenchement **manuel** du gate — acceptable si justifié ; (b) un déclenchement **automatique conditionné aux fichiers modifiés** — « une autre solution, potentiellement avec ses défauts » ; (c) **deux pipelines séparés**, l'un pour l'agent, l'autre pour le logiciel qui l'utilise — proposition d'un apprenant (Steve), retenue par le formateur comme la réponse la plus simple au brief. Ce dossier retient (b). Raison : (c) suppose **deux composants déployables** — un agent « back-end à lui tout seul » derrière une passerelle ; ici `/v2` est le code qui appelle le modèle, il n'y a pas de second composant. Séparer les pipelines exigerait d'abord de séparer l'architecture (un service agent, un contrat entre le routeur et lui, un second manifeste) — un chantier, pas un fichier de workflow. Le défaut de (b) est nommé en H19. Évolution documentée : si l'appel au modèle devient un service à part, le filtre de chemin devient le déclencheur naturel d'un second `ci.yml`, sans rien changer aux gates.

Quand un gate échoue : ce qu'on refait, ce qu'on ne refait pas. Question du formateur (18/09) : « si le gate d'évaluation échoue, refait-on tout le processus ? » Non — un gate rouge se rejoue **au niveau où il a échoué**, jamais depuis le début.

Le gate est rouge...	Ce qu'on refait	Ce qu'on ne refait pas	Pourquoi c'est peu coûteux
sur le poste du développeur — la chaîne se rejoue localement (section 3.6), gate d'évaluation compris si on le choisit	corriger, relancer	rien d'autre : ni push, ni PR	12 appels au modèle, bien sous 1 € (section 3.2) : c'est pour cela qu'il est permis si tôt
sur la pull request — job 1 à chaque push, job 2 par filtre de chemin	corriger sur la même branche , pousser ; les checks rejouent d'eux-mêmes, la revue automatisée recommande le nouveau diff	ni nouvelle branche, ni nouvelle PR, ni relecture reprise de zéro	c'est là que le filtre est fait pour attraper un changement de comportement — avant la fusion
à la fusion — jobs 1 et 2 rejoués sur le code fusionné	une nouvelle branche courte → PR → fusion : oui, la boucle entière	rien en production : aucun artefact construit, aucun tag posé, le tag précédent continue de servir — <code>main</code> est rouge, la production ne l'est pas	ce cas n'arrive que si <code>main</code> a bougé entre le vert de la PR et la fusion, ou si le changement est passé par un fichier hors filtre (H19) ; les deux niveaux précédents existent pour le rendre rare

La règle : **on ne refait jamais la chaîne entière à la main ; on rejoue le niveau qui a échoué**. L'alternative — ne jouer le gate payant qu'après la fusion — est exactement ce qui impose la boucle entière à chaque échec ; ce dossier n'a pas cette forme.

3.1 Comment lire le schéma

Tout en haut, encadré violet : **avant la fusion** — poste dev → branche → pull request → checks (détaillé en 3.0). En dessous, colonne de gauche : **la chaîne** (`11mops.yml`, GitHub Actions), déclenchée par une fusion sur `main` et rien d'autre. Six étapes ; les deux premières sont des **gates bloquants** — un losange orange, et une branche rouge « livraison bloquée » quand la réponse est non. Colonne de droite : **la production** — le routeur, le conteneur v1 figé, les deux conteneurs v2 (stable, canary), le fournisseur de modèle, et les signaux émis par requête. En bas, en gris pointillé : **le Chantier 2**, pas conçu ici, mais dont les points de raccord sont déjà dessinés — deux entrées vers la chaîne, deux sorties depuis la production. Le code couleur est en légende.

3.2 Les six étapes de la chaîne

#	Étape	Ce qui entre	Ce qui sort	Ce qui bloque
1	Tests (gate) — MOCK=on , LLM factice, gratuit	le code fusionné	rapport de tests	un test rouge : tests unitaires (score composite, courbe de calibration, note F1, analyse des types de clauses — sans modèle), contrat v2, erreurs explicites, rejeu de <code>scripts/client_v1.py</code> contre v1, tests d'intégration de la remédiation (les signaux sont bien émis sur <code>/v1</code>)
2	Gate d'évaluation (gate) — vrai modèle	le candidat + <code>attendus.json</code> 1 version N (12 contrats) + <code>thresholds.yaml</code> 1	note par contrat (F1 sur les types de clauses) ; courbe de calibration → erreur de calibration ; <code>calibration_vN.json</code> ; latence p95 et coût moyen mesurés sur les 12 contrats — les trois longs sont le vrai test	une note sous le <code>seuil_note</code> de son contrat (0,75 ; 0,8 pour les longs) ; erreur de calibration au-dessus du seuil ; p95 > 8 s ou coût moyen > 0,15 € par analyse — les contraintes de la direction juridique, écrites telles quelles dans <code>thresholds.yaml</code> : une release exacte mais trop lente ou trop chère ne part pas ; un cas obligatoire en échec (<code>xfail strict</code> : jamais abaisser le seuil, jamais retirer le cas). Coût du gate : 12 appels au modèle réel par fusion — à mesurer, attendu bien sous 1 €
3	Build	le code, le Bundle <code>models/v2/config.yaml</code> (modèle, prompt, paramètres dont la température), la courbe de calibration	une image Docker avec le manifeste embarqué (section 4) ; aucun secret dans l'image (section 3.7)	échec de construction
4	Tag	VERSION (semver) + <code>git_sha</code>	un tag git et le même tag sur l'image	tag déjà existant (une version ne se reconstruit jamais)
5	Push	l'image taguée	l'artefact immuable dans le registre (GHCR)	registre injoignable
6	Déploiement canary	le tag	<code>set_weight(nouveau_tag, 10 %)</code> dans <code>weights.json</code> ; le routeur lit le nouveau poids	<code>GET /v2/health</code> du nouveau conteneur en échec → poids laissé à 0 %

Deux règles transversales : **aucune étape ne s'exécute à la main** (le seul déclencheur est la fusion) ; **aucune étape ne reconstruit un artefact existant** (un tag est définitif, un rollback redéploie, il ne rebâtit pas).

3.3 « Artefact étiqueté » — ce que le brief demande, ce que la chaîne produit

Le brief exige qu'une fusion produise « une version étiquetée (code + modèle + config) ». Ici, l'artefact est **une image Docker immuable**, dont le tag est le `release_version`, et qui embarque un **manifeste** (section 4) listant le commit, l'identifiant daté du modèle, la version du prompt, l'empreinte de la configuration, la version de calibration et son score, la version du golden dataset. On ne possède pas les poids du modèle (API externe) : ce qui est versionné est **l'enveloppe** qui l'entoure, et c'est elle qui fait qu'un comportement est reproductible et attribuable. Le code fourni a déjà cette notion — le **Bundle**, `models/vX/config.yaml` (version, modèle, paramètres) : le manifeste l'enveloppe et y ajoute ce que le fichier de configuration ne peut pas savoir de lui-même (commit, note d'évaluation obtenue, version du jeu de référence, horodatage) ; il ne le remplace pas.

3.4 La production — où v1 ne casse jamais

- **Le routeur** (passerelle FastAPI maison) est le seul point d'entrée. /v1 est **toujours** envoyé au conteneur v1 ; /v2 est réparti entre v2 stable et v2 canary selon weights.json . Le routeur écrit une Mesure par requête (route, tag servi, statut, latence) dans le même ops/metrics.jsonl que les services — c'est lui qui rend la part v1/v2 mesurable sans rien ajouter à v1.
- **Le conteneur v1 est figé** : jamais redéployé par cette chaîne, seulement instrumenté — et il l'est déjà (spans, journaux, Mesure, depuis la remédiation) ; on y ajoute l'en-tête X-Mardik-Release . L'exigence « v1 ne casse jamais » est ainsi **structurelle** : elle ne dépend pas de la prudence d'un déploiement, il n'y a pas de déploiement.
- **Deux conteneurs v2** cohabitent : le stable (tag N) et le canary (tag N+1). La promotion du canary est l'affaire du Chantier 2 ; ici, la chaîne pose 10 % et s'arrête.
- **Les signaux** naissent ici, par requête, **dans le format fourni par la remédiation** : une Mesure dans ops/metrics.jsonl (ts, version, route, latence_ms, erreur, score, cout_eur, appels_llm, tokens, tronque) — la seule source lue par le tableau de bord (ops/dashboard.py) et la surveillance (ops/deploy.py::surveiller) fournis ; des spans OpenTelemetry (analyse.requete → llm.appeal) exportés en **OTLP** vers le collecteur choisi au Chantier 2 ; des journaux structurés. La v2 et le routeur émettent les mêmes signaux que la v1, plus le score que la v1 laisse vide : une version qui n'écrit pas là est invisible, donc impilotable. Le Chantier 2 lit ces signaux ; il ne les fabrique pas.

Pas d'environnement de staging, et pourquoi. La pratique de référence — une fusion sur main déploie en staging, un tag déploie en production, le même paquet partout — suppose un trafic capable de révéler quelque chose en staging. À vingt requêtes par jour, un environnement intermédiaire en recevrait deux, et ne prouverait rien. Ici, le canary à 10 % **sur le trafic réel** tient ce rôle : c'est le même artefact immuable (une image, un tag) qui passe de la chaîne au canary puis au stable — la règle « même paquet partout » est respectée sans environnement supplémentaire. Ce que l'on perd, dit franchement : aucun essai n'a lieu hors du trafic des juristes ; les 10 % du canary sont des requêtes réelles. Si le volume change d'ordre de grandeur, un staging s'insère entre les jobs 5 et 6 sans toucher aux gates.

3.5 Points de raccord avec le Chantier 2 — dessinés, pas conçus

Sens	Point de raccord	Qui l'actionne	Ce que ça change
vers la chaîne	redploy(previous_tag)	un humain , un clic (arbitrage du CTO : le rollback n'est pas automatique)	le routeur renvoie 100 % de /v2 vers un tag déjà validé ; rien n'est reconstruit ; l'événement est tracé
vers la chaîne	set_weight(tag, pct)	la boucle de promotion pilotée par les métriques (Ch2)	10 → 50 → 100 %, seulement si les critères tiennent
depuis la production	signaux par requête	le tableau de bord et le détecteur de dérive (Ch2)	rien ici — lecture seule
depuis la production	cas à faible confiance → golden dataset version N+1, par pull request	la boucle d'enrichissement (Ch2)	le gate d'évaluation rejoue ces cas dès cette PR (filtre de chemin, section 3.0), puis à la fusion suivante, et recalibre

Ces quatre raccords sont les seules frontières entre les deux chantiers. Le Chantier 2 ne touchera ni au contrat (section 2), ni à la chaîne (ce schéma) : il les pilote.

Deux choses que le Chantier 2 devra fixer, mais que ce schéma anticipe. **La fenêtre d'observation** : 400 contrats par mois, c'est une vingtaine de requêtes par jour ; une fenêtre comptée en requêtes se compte donc en jours — la règle sera « N requêtes ou D jours, la première atteinte », la démonstration se faisant sur le trafic simulé fourni (make traffic, ops/drift_proxy.py). **L'asymétrie entre avancer et reculer** : la

promotion est automatique et pilotée par les métriques (avancer quand les chiffres tiennent est réversible et à faible regret) ; le retour arrière est une décision humaine (un incident se juge, il ne se seuille pas) — arbitrage du CTO, retenu tel quel.

3.6 Ce qui prouve la chaîne

- Test d'acceptance 3 — le jeu de `client_v1.py` est le job 1, bloquant : si v1 casse, rien ne part.
- Test d'acceptance 4 — une fusion sur `main` produit un tag git, une image du même tag dans GHCR, et `GET /v2/version` renvoie ce tag ; X-Mardik-Release l'atteste à chaque requête.
- Test d'acceptance 5 — un golden dataset contenant un cas volontairement cassé fait échouer le job 2 : la chaîne s'arrête avant le build. Cette démonstration se prépare en **falsifiant une case**, pas en l'affirmant.
- Les deux contraintes de la direction juridique — $p95 < 8\text{ s}$, $< 0,15\text{ €}$ par analyse — sont des **critères du même gate**, mesurés sur les 12 contrats à chaque fusion : une release exacte mais trop lente ou trop chère ne part pas, et le manifeste enregistre les valeurs obtenues.
- La chaîne se rejoue localement (`act` ou un script équivalent) avant toute fusion : un développeur peut voir un gate rouge sans attendre le serveur d'intégration.
- La boucle de correction au niveau de la PR (section 3.0) se démontre en poussant un prompt volontairement mauvais sur une branche : job 2 rouge sur la PR ; correctif poussé **sur la même branche** ; job 2 vert — sans seconde PR, sans fusion.
- Le filtre de chemin du job 2 sur une PR (section 3.0) se vérifie avec deux PR de test : l'une qui ne touche que `README.md` (job 2 absent des checks), l'une qui touche `prompts/analyse-v2.0.txt` (job 2 présent) — capture des deux jeux de checks GitHub.

3.7 Où vivent les secrets — et correspondance avec `docs/exploitation.md`

Secrets. Deux secrets existent : la clé du fournisseur LLM et la clé d'API de `/v2` (hors démonstration). En intégration continue, ils sont des *secrets GitHub Actions*, injectés au job d'évaluation et jamais au job de tests (`MOCK=on` n'en a pas besoin). À l'exécution, ce sont des variables d'environnement du conteneur (fichier `.env` local non versionné, variables d'environnement sur le serveur). Ils ne sont **jamais** dans l'image, le manifeste, le registre, `weights.json` ni les journaux — les messages d'erreur de la section 2.7 n'exposent rien d'interne. Le manifeste enregistre *quel* modèle, jamais *comment on s'y connecte*.

Le runbook fourni. `docs/exploitation.md` est un livrable à rédiger, en sept sections ; il se remplit depuis ce dossier, sans rien écrire deux fois :

exploitation.md	Section de ce dossier
1. Qu'est-ce qu'une version ?	2.1 (deux horloges) · 3.3 (enveloppe, Bundle) · 4 (manifeste)
2. Schéma d'étiquetage	3.2 (jobs 4–5) · 4
3. Chaîne de livraison, MOCK vs vrai modèle	3.2 · 3.6
4. Déploiement progressif	3.4 · 3.5 (fenêtre, asymétrie)
5. Procédure de rollback — une commande, qui a le droit	3.5 (<code>redeploy(previous_tag)</code>) · Chantier 2 pour le journal
6. Surveillance et seuils	Chantier 2
7. Preuve d'exécution	Chantier 2 (<code>make traffic MODE=derive-score</code>)

4. Le manifeste de version

Question : après un changement de prompt ou de modèle, quand et où ce changement est-il retesté ?

Réponse courte : dès la pull request qui le touche (filtre de chemin sur `models/v2/config.yaml`, `prompts/`, le code d'analyse, `attendus.jsonl`, `thresholds.yml` — section 3.0) — pas seulement à la fusion, pour ne pas

découvrir l'échec après coup — puis de nouveau à la fusion sur le code réellement fusionné (section 3.2, job 2). Le manifeste ci-dessous est ce qui rend cette double vérification traçable a posteriori.

4.1 Deux couches : le Bundle fourni, et ce que le manifeste y ajoute

Une version livrable est « code + modèle + configuration » (note du CTO). Le code fourni porte déjà les deux derniers termes : le **Bundle**, `models/v2/config.yaml`, chargé par `Bundle.charger("v2")` exactement comme la v1 charge le sien. Le manifeste ne le remplace pas ; il l'**enveloppe** et y ajoute ce qu'un fichier de configuration ne peut pas savoir de lui-même — d'où il vient, ce qu'il a obtenu, quand.

Couche 1 — le Bundle (`models/v2/config.yaml`, versionné en git)

Champ	Contenu	Pourquoi il est là
<code>version</code>	<code>v2.0.0</code>	identité lisible, la même que le tag git et le tag de l'image
<code>modele</code>	identifiant daté du fournisseur	un « même » modèle mis à jour par le fournisseur est un autre modèle ; sans la date, deux comportements différents porteraient le même nom
<code>prompt + prompt_version</code>	le texte du prompt (fichier <code>prompts/analyse-v2.0.txt</code>) et son étiquette	un prompt changé change le comportement : c'est une release, pas un détail
<code>parametres</code>	<code>contexte_max_caracteres</code> , <code>temperature</code> (0 pour l'évaluation et la production), <code>max_tokens</code> , <code>timeout_s</code> , nombre de réessais	la température oubliée a coûté deux heures à la classe ; ici elle est écrite, versionnée, empreintée
<code>types_clauses</code>	le vocabulaire des 14 types	le langage partagé, figé par version

Couche 2 — le manifeste (`manifest.json`, produit par le job 3, embarqué dans l'image)

Champ	Contenu	Ce qu'il prouve
<code>release_version</code>	<code>v2.0.0</code>	= tag git = tag de l'image = <code>version</code> du Bundle : trois noms, une seule chose
<code>git_sha</code>	le commit fusionné	le code exact — <code>git show <sha></code> retrouve tout, dont le Bundle
<code>model_id</code>	recopié du Bundle	quel modèle a répondu, à quelle date de version du fournisseur
<code>prompt_version</code>	recopiée du Bundle	quel prompt
<code>config_hash</code>	SHA-256 de <code>config.yaml</code> (prompt et paramètres compris)	deux manifestes au même <code>config_hash</code> ont le même comportement attendu ; un hash différent dit <i>quelque chose</i> a changé, même si personne ne l'a annoncé
<code>calibration_version + eval_score</code>	<code>cal-v1</code> ; la note moyenne et la note minimale sur les 12 contrats, l'erreur de calibration	le gate a réellement tourné, et avec quel résultat — pas seulement « il existe »
<code>golden_dataset_version</code>	<code>attendus-v1</code> (empreinte de <code>attendus.jsonl</code>)	contre quelle vérité la note a été obtenue ; quand le Chantier 2 enrichit le jeu, ce champ change
<code>build_timestamp</code>	ISO 8601, UTC	quand ; l'ordre des versions ne se devine pas au numéro seul

```
{
  "release_version": "v2.0.0",
  "git_sha": "3f9c1e7",
  "model_id": "<fournisseur>-<modele>-2026-06-01",
  "prompt_version": "analyse-v2.0",
  "config_hash": "sha256:7b1e...",
  "calibration_version": "cal-v1",
  "eval_score": { "note_moyenne": 0.87, "note_min": 0.81, "erreur_calibration": 0.04 },
  "golden_dataset_version": "attendus-v1",
  "build_timestamp": "2026-09-23T14:02:11Z"
}
```

Ce que le manifeste ne contient **jamais** : une clé, une URL de fournisseur avec identifiant, un secret quelconque (section 3.7). Il dit *quel* modèle, pas *comment on s'y connecte*.

4.2 Où il vit — cinq endroits, une seule source

1. **Dans l'image**, à la racine de l'application (/app/manifest.json), plus les étiquettes OCI de l'image (org.opencontainers.image.version , .revision , .created) — un docker inspect suffit à identifier une image sans la lancer.
2. **GET /v2/version** renvoie ce fichier tel quel : la version qui répond se décrit elle-même.
3. **X-Mardik-Release** sur chaque réponse, /v1 comprise : la version qui a servi *cette* requête — la preuve demandée par le brief, à la granularité de la requête.
4. **Le registre** (fourni, lu par le tableau de bord) : la liste des versions et celle qui sert le trafic, avec le poids canary.
5. **Chaque Mesure** de ops/metrics.jsonl porte version : chaque ligne de métrique est attribuable à une release.

Le fichier dans l'image est la source ; les quatre autres le recopient ou y pointent. Rien n'est saisi à la main.

4.3 Deux horloges, un numéro : ce qui change quoi

release_version suit vMAJOR.MINOR.PATCH, lu dans le fichier VERSION :

Ce qui change	Incrémente	Ce que la chaîne rejoue	Ce qui le rend vérifiable
Le contrat (/v2 → champ retiré, sens changé, code d'erreur changé)	MAJOR → nouvelle route /v3 , /v2 maintenu	tout	l'URL
Le prompt · le modèle du fournisseur (nouvel identifiant daté) · un paramètre (température, plafond) · une recalibration · un jeu de référence enrichi	MINOR	le gate d'évaluation, obligatoirement	prompt_version , model_id , config_hash , calibration_version , golden_dataset_version différent entre les deux manifestes
Le code sans effet attendu sur le comportement (correctif, refactorisation, journaux)	PATCH	le gate d'évaluation quand même — « sans effet attendu » est une hypothèse, le gate la vérifie	git_sha diffère, config_hash ne diffère pas

La règle qui protège tout le reste : **aucune version ne se reconstruit**. Un tag posé est définitif ; corriger, c'est produire la version suivante ; revenir, c'est redéployer une version antérieure qui a déjà passé le gate.

Un changement MINOR déclenche le gate d'évaluation **deux fois** : sur la pull request qui le touche (section 3.0, avant la fusion — l'auteur voit l'échec tout de suite, sans attendre) et à la fusion elle-même (section 3.2,

job 2, systématique — sur le code réellement fusionné, qui peut différer de celui de la PR). Un changement PATCH ne déclenche le job 2 avant la fusion que s'il touche l'un des chemins surveillés (rare pour un correctif qui n'y touche pas) ; il le déclenche toujours à la fusion.

4.4 Retrouver, à 3 h du matin, ce qui tourne

La question du runbook (`exploitation.md` , § 2) : *comment retrouver le commit et la note d'évaluation d'une version en production ?* — `GET /v2/version` sur la production → `git_sha` et `eval_score` → `git show <sha>` pour le code et le Bundle, le rapport du gate archivé avec le tag (artefact de l'exécution `llmops.yml`) pour le détail par contrat. Trois commandes, aucune mémoire humaine.

4.5 Intégrité — ce qui est fait, ce qui est dit

Le manifeste est empreinté (SHA-256) et l'empreinte de l'image (digest) est écrite au registre au job 5 : un manifeste modifié après coup ne correspond plus à son image. La signature de l'image (cosign) est la marche suivante ; elle est **hors de ce qui est promis** ici — solo, temps court — et sera faite si le temps le permet, jamais annoncée avant.

4.6 Choix d'outils non triviaux — ce qui est retenu, ce qui est écarté, pourquoi

Les outils déjà familiers (Python, FastAPI, Docker, GitHub Actions, pytest) ne sont pas justifiés ici. Ceux qui suivent le sont, parce qu'une alternative existait.

Retenu	Alternative écartée	Pourquoi	Section
Routeur FastAPI maison , poids lus en direct dans <code>weights.json</code>	Traefik ou Caddy, fichier de configuration régénéré puis rechargé	un seul point de passage où la version servie et la part de trafic se journalisent sans rien ajouter à v1 ; une quarantaine de lignes défendables ligne à ligne ; un changement de poids ne redémarre rien	§3.4
Régression isotonique pour calibrer le score	régression de Platt ; score brut du modèle	monotone et sans hypothèse de forme — lisible sur 168 décisions ; le score brut n'est pas calibré, il dit 0,8 et se trompe une fois sur deux	§2.5
GHCR comme registre d'images	Docker Hub	même identité que le dépôt et la chaîne (jeton GitHub Actions, aucun compte supplémentaire) ; l'image porte le tag git	§3.2 (job 5)
Erreurs <i>problem details</i> (RFC 9457)	format d'erreur ad hoc	un standard existant, une seule forme, des code stables que le client peut tester	§2.7
Filtre de chemin GitHub Actions pour le gate payant	déclenchement manuel ; deux pipelines (agent / logiciel)	le manuel est acceptable mais n'automatise rien ; deux pipelines supposent deux composants déployables, absents ici ; le filtre est l'option que le formateur a nommée « avec ses défauts » — défaut écrit en H19	§3.0
Revue automatisée de PR — Claude Code GitHub Action (jeton d'abonnement <code>CLAUDE_CODE_OAUTH_TOKEN</code> , secret du dépôt)	auto-revue seule ; GitHub Copilot code review ; la même action sur un backend cloud (Bedrock, Vertex, Foundry)	seul sur le projet : un second regard avant la fusion ; l'outil déjà utilisé pour développer, sans clé d'API ni compte de plus — le backend cloud n'apporte rien ici ; Copilot reste le repli si le jeton n'est pas permis sur le dépôt	§3.0
Docker Compose (démonstration), Azure (cible)	Kubernetes	trois conteneurs, une personne, un fichier ; tout est conteneur — le lieu change, pas l'architecture	H16
Signature cosign	—	différée : hors de ce qui est promis, faite si le temps le permet, jamais annoncée avant	§4.5

5. Hypothèses et limites

Un dossier qui ne dit pas ce qu'il suppose ni ce qu'il ne fait pas n'est pas crédible. Voici les deux listes.

5.1 Ce qui est un fait, et d'où il vient

Fait	Source
p95 < 8 s contrats longs compris ; < 0,15 € par analyse ; ~400 contrats/mois ; 3 juristes ; texte collé ; <code>client_v1</code> à ne pas refaire ; score par clause et global	<code>ressource/docs/besoin_client.md</code>
Contrat <code>/v1 : {"texte"} → {"clauses", "modele", "version", "tronque"}</code> , 422/503 ; coupure à 16 000 caractères ; Bundle <code>models/vX/config.yaml</code>	<code>ressource/api_v1.py</code>
12 contrats de référence, 14 types de clauses, <code>seuil_note</code> 0,75 / 0,8	<code>ressource/attendus.jsonl</code>
Télémetrie fournie : spans OTel, journaux structurés, <code>Mesure → ops/metrics.jsonl</code> , seule source du tableau de bord et de la surveillance ; <code>MOCK=on</code> ; proxy de dérive ; tests d'intégration verts	<code>ressource/docs/schema_remediation.md</code> , <code>docs/exploitation.md</code>
Rollback manuel, un clic, par un non-développeur ; un conteneur aujourd'hui, plusieurs si justifié ; documents collés par les métiers ; l'équipe propose les KPI ; OTLP pour les traces ; seuils par baseline mesurée	réponses du CTO / formateur en séance, 16/09 (<code>Mardik-BRIEF-original.md</code> , annexes B et D)

5.2 Ce qui est supposé — et ce qui se passe si c'est faux

#	Hypothèse	Si elle est fausse
H5	Plafond <code>max_caracteres</code> $\approx 200\,000$ (≈ 50 pages), au-dessus des 40 pages du besoin	une ligne de configuration, versionnée ; le contrat ne change pas
H6	Bornes du libellé : <code>haute</code> $\geq 0,8 \cdot$ <code>moyenne</code> $[0,5 ; 0,8[\cdot$ <code>basse</code> $< 0,5$	ajustées au Chantier 2 sur les distributions observées, en configuration versionnée — c'est prévu, pas subi
H8	Un appel au modèle par requête ; le gate d'évaluation coûte 12 appels par fusion	si le coût du gate dépasse ce qu'on accepte, on évalue sur un sous-ensemble tournant, jamais sur un seul contrat
H9	Les textes des 12 contrats arrivent avec le dépôt	sinon : contrats synthétiques au format de <code>exemple_de_contrat.txt</code> , annoncés comme tels, remplacés dès réception
H10	Fenêtre canary « N requêtes ou D jours, la première atteinte »	c'est le Chantier 2 qui fixe N et D ; la mécanique (poids dans <code>weights.json</code>) ne bouge pas
H11	Le canary répartit deux releases de v2 derrière le routeur ; v1 figée et vivante ; « part v1/v2 » = part des requêtes <code>/v1</code> vs <code>/v2</code>	si le formateur entend « migration des clients v1 vers v2 », le routeur garde la même structure ; seule la règle de poids change (par client au lieu de par requête)
H12	Seuils initiaux = <code>seuil_note</code> fournis + baseline v1 mesurée sur les 12 contrats ; latence et coût = les contraintes du client telles quelles (8 s, 0,15 €), qui ne sont pas des hypothèses	le principe (« mesuré, pas décidé ») tient quelle que soit la baseline retenue
H14	Les trois boucles sont construites ; la démonstration en garantit deux	si deux suffisent, l'enrichissement est livré mais non démontré ; rien n'est retiré
H15	Frontend : page unique, sans authentification en démonstration, affichant clauses, niveau de certitude, version	un lien public demandé dès la conception → hébergement à prévoir plus tôt (Hetzner, déjà disponible)
H16	Démonstration en Docker Compose local ; Azure = cible portable	tout est conteneur : le lieu change, pas l'architecture
H17	Signature cosign « si le temps le permet »	absente = dite absente, jamais présentée comme faite

#	Hypothèse	Si elle est fausse
H18	Un appel unique sur 40 pages tient sous 8 s p95 avec une sortie courte (14 types + confiances)	la seule hypothèse qui peut faire changer une décision : le gate d'évaluation la vérifie à chaque fusion (critère bloquant, section 3.2) ; si elle tombe, on change le modèle ou le plafond — pas le contrat
H19	Le filtre de chemin (<code>models/v2/config.yaml</code> , <code>prompts/</code> , le code d'analyse, <code>attendus.jsonl</code> , <code>thresholds.yml</code>) couvre tous les changements qui modifient le comportement du modèle ou ce que le gate mesure (section 3.0)	si un changement de comportement passe par un autre fichier (ex. une dépendance du fournisseur LLM mise à jour), le gate ne se déclenche pas sur la PR — mais se déclenche quand même à la fusion (job 2 systématique, section 3.2) : seule la détection précoce est perdue, jamais la garantie

5.3 Hors périmètre (rappel)

Traduction en anglais · chatbot · découpage au-delà du plafond · dépôt de fichiers PDF · toute modification fonctionnelle de v1 · le tableau de bord, les trois boucles et le journal de pilotage (Chantier 2, après validation de celui-ci). Chacun avec sa raison en section 1.

5.4 Ce que ce dossier ne fait pas, et ses limites

- **Rien n'est codé, rien n'est mesuré.** Le dépôt est inaccessible : les textes des contrats, `client_v1.py` , `traffic_sim.py` , le registre et l'éventuelle gateway sont inconnus. Chaque chiffre de ce dossier vient d'une source citée ou est marqué hypothèse ; aucun n'a été mesuré par nous.
- **Le score n'est calibré que sur le domaine du jeu de référence** : contrats fournisseurs en français, 12 cas, 168 décisions. C'est petit pour une courbe de calibration — assez pour une première version, pas pour une garantie. `calibre: false` dit la frontière ; l'enrichissement du Chantier 2 la repousse.
- **La note F1 traite tous les types de clauses à égalité** : manquer « droit applicable » pèse autant que manquer « durée ». Une pondération par gravité métier est possible ; elle demande une décision des juristes, pas de l'équipe.
- **Le contrat ne renvoie ni extraits ni positions** de clauses (le besoin exprimé est la liste des types) ; c'est l'extension la plus probable, prévue sans rupture par la forme actuelle de `clauses[]` .
- **Vingt requêtes par jour** : un canary réel se compte en jours ; la démonstration se fait sur trafic simulé — une démonstration n'est pas la production, et le dossier ne prétend pas le contraire.
- **La promotion automatique est décidée, pas conçue** : ses critères, sa fenêtre et sa trace sont le Chantier 2.
- **Aucune revue externe** de ce dossier avant le formateur : une seule personne l'a écrit et relu.

6. Critères RNCP couverts

Les compétences visées par le topic (annexe A du brief) et ce que ce dossier apporte à chacune. « Conçu » n'est pas « prouvé » : la colonne *statut* le dit.

Compétence	Où dans ce dossier	Ce que le jury peut vérifier	Statut
C14 — Analyser le besoin d'un commanditaire intégrant un service d'IA	\$1, \$5.1	besoin, valeur, utilisateurs, contraintes chiffrées, hors périmètre argumenté, sources citées	conçu
C15 — Concevoir le cadre technique d'une application	\$2, \$3, \$4	contrat, chaîne, versionnage, secrets ; choix d'outils et alternatives écartées avec leurs raisons (\$4.6, <code>CONTINUITE.md</code>)	conçu

Compétence	Où dans ce dossier	Ce que le jury peut vérifier	Statut
C9 — Développer une API exposant un modèle d'IA	\$2	contrat en schémas, erreurs explicites, deux horloges, idempotence, score calibré	conçu ; prouvé par <code>tests/test_v2_*.py</code> au développement
C13 — Créer une chaîne de livraison continue (MLOps)	\$3.2, \$3.3, \$4	gates bloquants dont le gate d'évaluation à seuils versionnés, artefact immuable, manifeste	conçu ; prouvé par <code>11mops.yml</code> et un tag au développement
C19 — Créer un processus de livraison continue	\$3.2 (jobs 3–6), \$3.5	build → tag → push → canary ; rollback = redéploiement d'un tag	conçu ; prouvé par une exécution de la chaîne
C18 — Automatiser les phases de tests du code source	\$2.8, \$3.2 (job 1)	tests de contrat et d'erreurs, rejeu de <code>client_v1.py</code> , tests d'intégration, <code>MOCK=on</code>	conçu ; prouvé par le rapport de tests
C17 — Développer les composants techniques et les interfaces	\$3.4 (routeur), H15 (frontend)	routeur maison, page cliente affichant clauses, niveau de certitude et version	conçu ; prouvé au développement
C16 — Coordonner la réalisation technique	processus chantier par chantier ; <code>ETAT.md</code> , <code>CONTINUITE.md</code> , <code>CHANGELOG.md</code>	décisions datées avec leur pourquoi, questions ouvertes, statuts à huit niveaux	en cours
C11 — Monitorer un modèle d'IA	\$3.4 (signaux, Mesure)	les cinq signaux émis par requête dans le format fourni	préparé ici, livré au Chantier 2
C20 — Surveiller une application d'IA	\$3.5 (raccords), \$3.7 (runbook)	tableau de bord, dérive, seuils	Chantier 2
C21 — Résoudre les incidents techniques	\$3.5 (rollback), \$4.4 (retrouver ce qui tourne), <code>exploitation.md</code>	rollback en une commande, tracé ; diagnostic par <code>request_id</code> et manifeste	préparé ici, démontré au Chantier 2
C1 · C2 (transversales) — planifier, piloter	ce dossier et ses fichiers d'état	plan chantier par chantier, tâche principale, journal des décisions	en cours
C4 (transversale) — rechercher méthodiquement	\$5, annexe A	questions → hypothèses → faits sourcés → décisions, alternatives rejetées	conçu
C5 (transversale) — partager la solution	ce dossier, le runbook, le glossaire	un lecteur qui n'était pas dans la salle peut suivre chaque décision jusqu'à sa source	conçu
C8 (transversale) — interagir de façon constructive	annexes B et D du brief	questions posées au CTO, vocabulaire métier repris, arbitrages acceptés (rollback manuel)	en cours

Non visées par le topic mais touchées : C10 (intégration de l'API de modèle dans l'application), C12 (tests automatisés du modèle — le gate d'évaluation en est un).

Annexe A — Réponses aux questions de cadrage du brief (Chantier 1)

Le brief liste des questions « pour guider la réflexion » (les apprenants) et des questions de référence (le formateur). Une ligne par question, et la section qui la traite — pour qu'un lecteur vérifie que le dossier est parti des questions, pas des réponses.

Question (résumée)	Réponse en une ligne	Section
Que demande <code>besoin_client.md</code> ? fonctionnel vs contrainte ? hors périmètre ?	contrats longs + score par clause = fonctionnel ; 8 s, 0,15 € = contraintes ; traduction, chatbot, chunking, PDF = hors périmètre	\$1, \$5.3
Quel contrat pour <code>/v2</code> : entrées, sorties, erreurs ?	JSON texte ; 14 types avec présence et confiance, score global, version ; erreurs RFC 9457 avant tout appel	\$2.3, \$2.4, \$2.7
D'où vient le score de confiance, que signifie-t-il pour le client ?	auto-évaluation par type, calibrée sur <code>attendus.json1</code> ; libellé = règle de décision pour le juriste	\$2.5
Que signifie « score de confiance » pour un juriste ; quel terme métier ?	« niveau de certitude » : haute → valider, moyenne → vérifier, basse → relire	\$2.5
Comment les métiers décrivent-ils un contrat « mal traité » ; langage partagé ?	« contrat tronqué », « clauses de fin oubliées » ; vocabulaire des 14 types repris tel quel dans le contrat	\$1, \$2.1
Qu'est-ce qui a le droit de changer dans <code>/v1</code> ? <code>v1</code> « figée de chez figée » ?	rien de fonctionnel ; instrumentation seulement (déjà en place) + en-tête de version	\$1, \$3.4
Canary : deux releases ou migration des clients ?	deux releases de v2 derrière le routeur, v1 figée à côté — hypothèse H11, posée au formateur	\$3.4, \$5.2
Deux ou trois boucles ?	trois construites, deux garanties en démonstration	\$5.2 (H14)
M1 / M3 ?	supprimés volontairement par le formateur	annexe B du brief
Qu'est-ce qu'une version livrable : code + poids + configuration ?	pas de poids (API externe) : Bundle + manifeste = l'enveloppe	\$3.3, \$4.1
Comment l'étiqueter ; comment savoir quelle version tourne ?	semver = tag git = tag image ; GET <code>/v2/version</code> , X-Mardik-Release, registre, <code>Mesure.version</code>	\$4.2, \$4.3
Quelles étapes, lesquelles bloquent ? tests unitaires, d'intégration (remédiation), gate d'évaluation ?	six jobs ; 1 (tests unitaires, contrat, erreurs, rejeu v1, intégration ; MOCK=on) et 2 (évaluation : note, calibration, latence p95 , coût ; vrai modèle) bloquent ; build, tag, push, canary ne jugent pas	\$3.2
Qu'est-ce qui dit que le build est correct ?	note par contrat $\geq$ <code>seuil_note</code> , erreur de calibration $\leq$ seuil, $p95 \leq 8$ s, $\text{coût} \leq 0,15$ €, tests verts — pas un avis	\$3.2, \$2.5
Sur quelle base les seuils initiaux (note minimale, latence max), provisoires ?	note : <code>seuil_note</code> fournis + baseline v1 mesurée ; latence et coût : les contraintes du client telles quelles ; le Chantier 2 ajuste, chaque ajustement = commit tracé	\$3.2, \$5.2 (H12)
Quelle mécanique canary : fraction initiale, routage, secrets ?	10 % ; routeur maison lisant <code>weights.json</code> ; secrets en variables d'environnement et secrets CI, jamais dans l'image	\$3.4, \$3.5, \$3.7
Comment se passe le rollback ?	une commande, un clic, un humain ; redéploiement d'un tag validé ; tracé	\$3.5, \$4.3
Quelle découpe (des documents) ?	aucune en v2.0 : appel unique, plafond explicite ; découpage = itération future	\$1, \$5.3
API et contrat : quelle entrée, quelle sortie ?	voir contrat	\$2
(18/09, retour individuel du formateur) Quelle stratégie de branche ; quels tests, où, pourquoi pas tous à chaque fois ?	pas de branche <code>dev</code> partagée, une branche courte par changement ; job 1 (gratuit) à chaque push, job 2 (payant) sur la PR seulement si elle touche modèle/prompt/config, le jeu de référence ou les seuils	\$3.0
(18/09) Quand reteste-t-on un changement de modèle ou de prompt ?	dès la PR qui le change (filtre de chemin), puis de nouveau à la fusion sur le code réellement fusionné	\$3.0, \$4.3

Question (résumée)	Réponse en une ligne	Section
(18/09) Comment présenter : où va-t-on avant le détail ?	chaque section technique (§2, §3, §4) ouvre désormais par « Question → Réponse courte »	§2, §3, §4
(18/09, retour du formateur sur le dossier envoyé) Si le gate d'évaluation échoue, refait-on tout le processus ?	non : rouge en local → relancer ; rouge sur la PR → correctif sur la même branche ; rouge à la fusion → nouvelle PR, mais rien n'a changé en production et ce cas est rare par construction	§3.0
(18/09, séance de classe) Un pipeline ou deux ? Un environnement de staging ?	un pipeline, gate conditionné aux fichiers (option validée par le formateur) ; deux pipelines = proposition de Steve, retenue par le formateur, qui suppose deux composants déployables — évolution documentée ; pas de staging à 20 requêtes/jour, le canary sur trafic réel en tient lieu	§3.0, §3.4

Les questions du Chantier 2 (signaux, traçabilité de bout en bout, stockage des traces, signaux de rollback, tableau de bord, seuils, « what steers the migration ») sont traitées dans le livrable du Chantier 2 ; ce dossier a seulement veillé à ce qu'elles restent possibles (§3.4, §3.5).

Glossaire

- **Score calibré** : score dont la valeur correspond à une exactitude mesurée sur un jeu de référence (0,8 → juste environ 80 % du temps).
- **Gate bloquant** : contrôle automatique de la chaîne qui empêche la livraison si un critère n'est pas atteint.
- **Version étiquetée** : livrable lié à une version unique et traçable (code + modèle + configuration), jamais reconstruite après étiquetage.
- **Canary** : envoi de la nouvelle version à une petite part du trafic, augmentée par paliers si les métriques tiennent.
- **Latence p95** : durée sous laquelle 95 % des requêtes aboutissent — la mesure de référence, pas la moyenne.
- **Chunking** : découpage d'un document en fragments analysés séparément puis recombinaison.
- **Instrumentation** : ajout de journaux et de mesures à un système sans changer ce qu'il fait.
- **Contrat d'API** : la forme figée des requêtes, des réponses et des erreurs d'un service — ce qu'un client peut supposer sans surprise.
- **Schéma Pydantic** : classe Python qui décrit et valide la forme d'une donnée ; ici, la source du contrat.
- **OpenAPI** : description standard et lisible par machine d'une API, générée ici depuis les schémas.
- **Horloge de version** : ce qui fait avancer un numéro de version ; le contrat et le comportement en ont deux différentes.
- **request_id** : identifiant unique d'une requête, présent dans chaque réponse et chaque journal — la clé de corrélation.
- **Idempotence** : propriété d'une opération qui, répétée avec la même clé, produit le même résultat sans effet supplémentaire (ici : sans second appel payé).
- **Réessai (retry) avec attente croissante** : nouvelle tentative après un échec transitoire, en attendant un peu plus à chaque fois.
- **Budget de latence** : délai total accordé à une requête, toutes tentatives comprises.
- **Problem details (RFC 9457)** : format standard d'une réponse d'erreur HTTP (`type`, `title`, `status`, `detail`).
- **Score composite** : score construit à partir de plusieurs composantes mesurables, avant calibration.
- **Régression isotonique** : méthode statistique qui apprend une correction monotone (croissante) entre un score brut et une exactitude observée.
- **Golden dataset** : jeu de cas de référence avec sorties attendues, versionné, contre lequel chaque release est notée.
- **Manifeste** : fichier embarqué dans la release qui liste exactement ce qu'elle contient (section 4).
- **GitHub Actions / job** : le moteur d'intégration continue de GitHub ; un *job* est une étape de la chaîne décrite dans `llmops.yml`.
- **Image Docker immuable** : paquet exécutable figé après construction ; on en produit une nouvelle, on ne modifie jamais l'existante.

- **Registre d'images (GHCR)** : entrepôt où les images taguées sont poussées puis tirées par la production.
- **Routeur / passerelle** : le service en façade qui décide, pour chaque requête, quel conteneur répond.
- **Poids de routage** : la part du trafic (en %) envoyée à chaque version ; ici dans `weights.json`.
- **Docker Compose** : description en un fichier de plusieurs conteneurs lancés ensemble (démon locale).
- **JSONL** : un objet JSON par ligne — le format des journaux par requête.
- **`xfail strict`** : marquer un test comme « échec attendu » qui reste rouge dans le rapport et redevient bloquant dès qu'il passe — l'inverse de baisser un seuil.
- **Bundle** : dans le code fourni, le dossier `models/vX/` et son `config.yaml` — la notion de version du code (modèle, prompt, paramètres).
- **`MOCK=on`** : mode fourni où l'application appelle un LLM factice et déterministe — tests gratuits, sans appel au fournisseur.
- **`seuil_note`** : la note minimale par contrat fixée dans `attendus.jsonl` (0,75 ; 0,8 pour les contrats longs).
- **Note (d'évaluation)** : exactitude de la liste de clauses rendue contre la liste attendue — ici le F1, moyenne harmonique de la précision et du rappel sur les types présents.
- **Mesure / `ops/metrics.jsonl`** : une ligne JSON par requête, écrite par la télémétrie fournie — la source du tableau de bord et de la surveillance.
- **Span** : un segment de trace OpenTelemetry (ici `analyse.requete` puis `llm.appel`), avec ses attributs.
- **OTLP** : le protocole de transport d'OpenTelemetry — les traces partent dans ce format, quel que soit le collecteur derrière.
- **structlog** : bibliothèque de journaux structurés (JSON) utilisée par la télémétrie fournie.
- **Proxy de dérive** (`ops/drift_proxy.py`) : composant fourni, entre l'application et le modèle, qui dégrade le trafic exprès (`DRIFT=latence|erreurs|score`) pour tester la détection.
- **Runbook** (`docs/exploitation.md`) : le document d'exploitation « qu'on lit à 3 h du matin quand la production dérive ».
- **Registre** : le relevé des versions existantes et de celle qui sert le trafic (détail connu à l'ouverture du dépôt).
- **Secret** : une valeur qui donne un accès (clé du fournisseur, clé d'API) — jamais versionnée, jamais dans une image.
- **Semver (`vMAJOR.MINOR.PATCH`)** : convention de numérotation — MAJOR pour un changement incompatible, MINOR pour un changement de comportement compatible, PATCH pour une correction.
- **`config_hash`** : empreinte SHA-256 du fichier de configuration ; deux empreintes égales = même configuration, à l'octet près.
- **Étiquettes OCI** : métadonnées standard d'une image de conteneur (`org.opencontainers.image.*`) lisibles sans la lancer.
- **Digest d'image** : empreinte SHA-256 du contenu d'une image ; elle change si un seul octet change.
- **Empreinte (hash)** : résumé de taille fixe d'un contenu, qui change dès que le contenu change — sert à prouver qu'un fichier n'a pas bougé.
- **Précision / rappel / F1** : part des clauses annoncées qui sont justes / part des clauses attendues qui ont été trouvées / leur moyenne harmonique — la note d'un contrat.
- **Baseline** : le système de référence mesuré (ici v1 sur les 12 contrats) que la version suivante doit battre ; celle qui le bat devient la nouvelle baseline.
- **Compétence C** : entrée du référentiel RNCP 37827 (ou du référentiel transversal) visée par le brief ; « conçu » ne vaut pas « prouvé ».